Solution: Depth First Search

Problem Statement

Write Recursive Approach for Depth First Search (DFS).

Given a graph, perform Depth First Search (DFS) traversal on the graph using a recursive approach.

Example 1:

Graph:

```
  1 -- 2
 / \    \
3   4 -- 5
```

Output: 1 2 5 4 3

Explanation: Starting from node 1, we visit its adjacent nodes in order: 2, 5, and 4. From node 4, we visit node 3.

Example 2:

Graph:

```
0 -- 1 -- 3
 \  |
   2
```

Output: 0 1 3 2

Explanation: Starting from node 0, we visit its adjacent nodes in order: 1 and 2. From node 1, we visit node 3.

Example 3:

Graph:

```
0 -- 1
|    |
3 -- 2
```

Output: 0 1 2 3

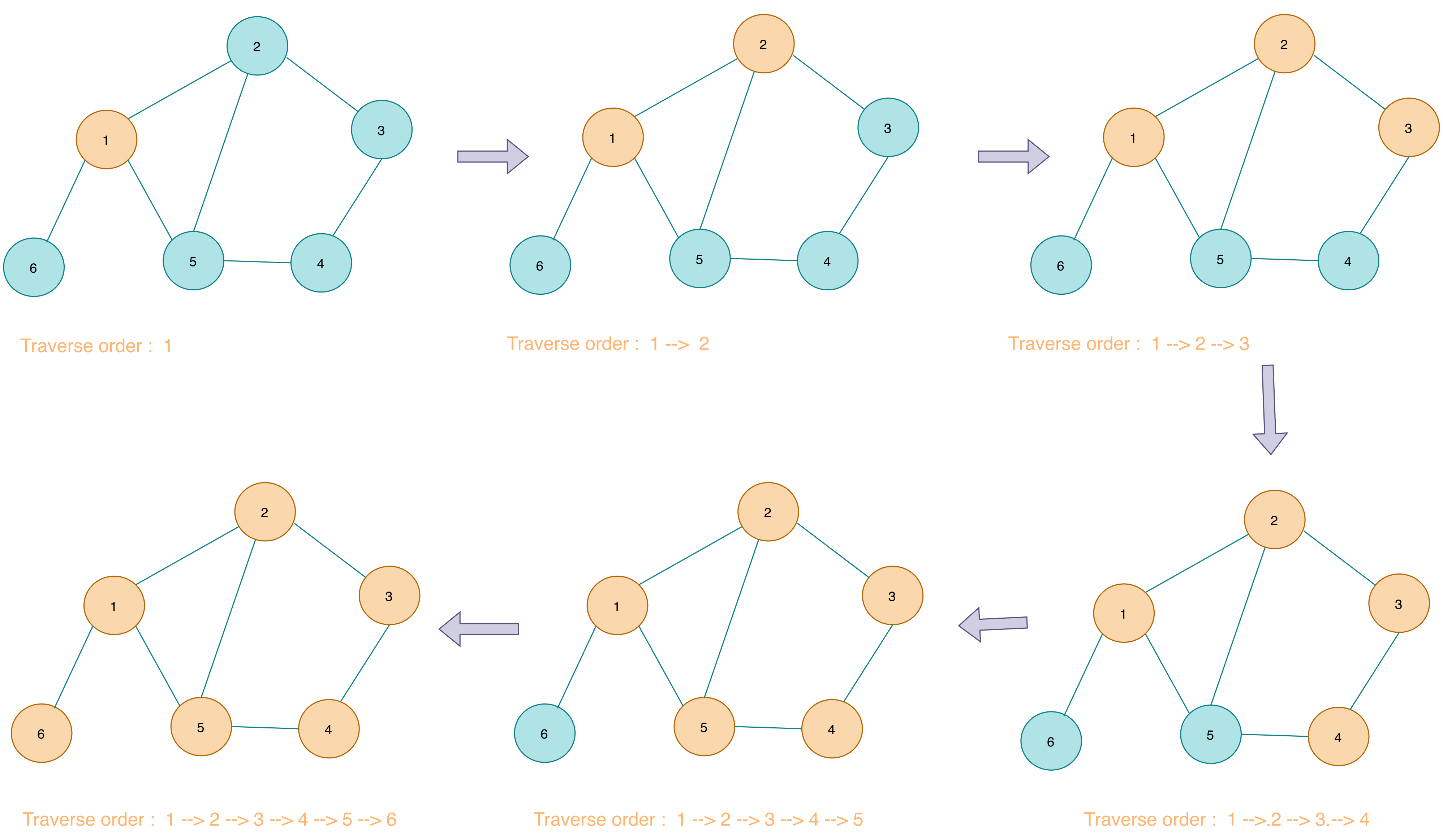
Explanation: Starting from node 0, we visit its adjacent nodes in order: 1 and 3. From node 1, we visit node 2.

Solution

The recursive DFS approach involves visiting each node of the graph and recursively visiting its adjacent nodes. The algorithm can be implemented as follows:

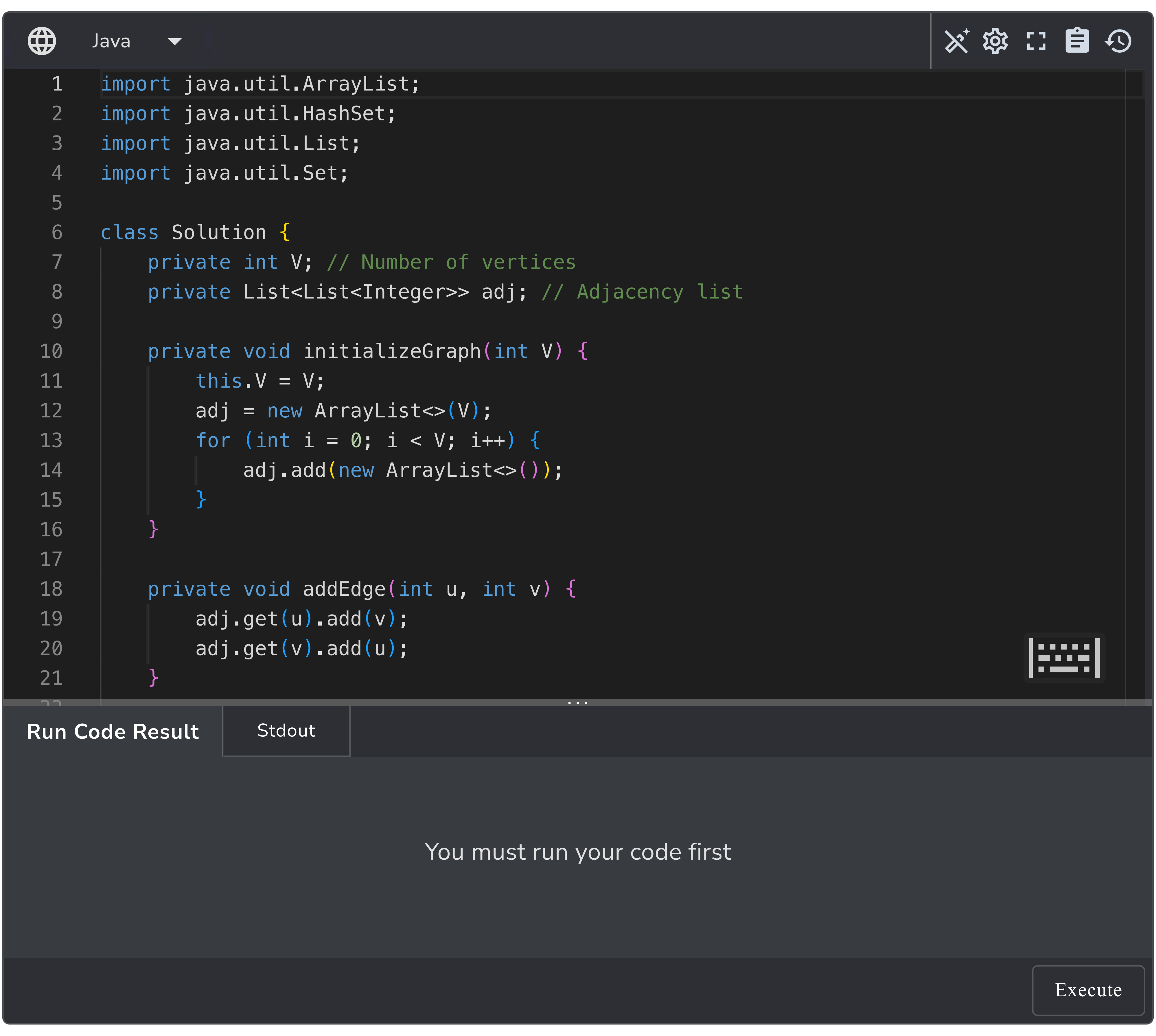
1. Create a recursive function, `dfs`, that takes a node and a visited set as parameters.
2. If the current node is not in the visited set, add it to the visited set and print its value.
3. Recursively call `dfs` on each unvisited adjacent node of the current node.

The base case for the recursive function is when the current node is already visited.


Traverse order : 1
Traverse order : 1 --> 2
Traverse order : 1 --> 2 --> 3
Traverse order : 1 -->.2 --> 3.--> 4
Traverse order : 1 --> 2 --> 3 --> 4 --> 5
Traverse order : 1 --> 2 --> 3 --> 4 --> 5 --> 6

Codes

Here is the code for this algorithm:



```java
import java.util.ArrayList;
import java.util.HashSet;
import java.util.List;
import java.util.Set;

class Solution {
    private int V; // Number of vertices
    private List<List<Integer>> adj; // Adjacency list

    private void initializeGraph(int V) {
        this.V = V;
        adj = new ArrayList<>(V);
        for (int i = 0; i < V; i++) {
            adj.add(new ArrayList<>());
        }
    }

    private void addEdge(int u, int v) {
        adj.get(u).add(v);
        adj.get(v).add(u);
    }
```

You must run your code first

Time and Space Complexity

The time complexity of the DFS algorithm using recursion is $O(V+E)$, where V is the number of vertices and E is the number of edges in the graph. This is because, in the worst case, we visit all the vertices and edges of the graph. The space complexity is $O(V)$, where V is the number of vertices, due to the space used for the visited set.